An Experimental Comparison of Sorting Algorithms: Bridging Theory and Real-World Performance

Omar Lekhal
Year 1, Group 2
Department of Computer Science
West University of Timişoara
omar.lekhal10@e-uvt.ro

April 2026

Abstract

Sorting is one of the most studied problems in computer science, yet the gap between theoretical complexity analysis and real-time performance remains a source of practical confusion. This paper presents the results of testing nine sorting algorithms (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Counting Sort, a Linked-List Merge Sort, and Python's built-in TimSort) across seven types of data and seven input sizes ranging from [50 to 10 000 000] elements. The results really showed the difference between $O(n^2)$ and $O(n \log n)$, also showing that: all three $O(n^2)$ algorithms exceed a 10-second hard timeout at $n = 10\,000$, while every $O(n \log n)$ algorithm handles the same size in under 20 ms. We also noticed that TimSort is incredibly fast on lists that are already sorted(sorting 10 000 000 already-sorted integers in 145 ms), Counting Sort was the fastest for big large integer inputs (74 ms at $n = 10^6$) but is inapplicable to floating-point or string data, and pure Python implementations of Merge Sort and Heap Sort are highly outperformed by TimSort. This paper provides a real world testing of the sorting algorithms using Python, and a simple guide for picking the most suitable sorting algorithm for different situations.

Contents

1 Introduction **3**

2 Formal Description of the Problem and Algorithms **4**
2.1 The Sorting Problem 4
2.2 Complexity Summary 4
2.3 Key Algorithmic Properties 5

3 Implementation and Experimental Setup **5**
3.1 Implementation Details 5

3.2 Benchmark Harness and Timeout Mechanism 6
3.3 Input Distributions and Sizes 6

4 Experimental Results and Analysis **7**
4.1 The $O(n^2)$ Performance Wall 7
4.2 Performance of $O(n \log n)$ Algorithms 7
4.3 Counting Sort and Data-Type Constraints 8
4.4 TimSort on Adaptive Profiles 8

5 Related Work **9**

6 Conclusions and Future Work **9**

1 Introduction

Motivation

Sorting is one of the most important braches in computing. From how databases find data to network routing tables, nearly every large-scale system relies on efficient ordering at its core. Textbooks such as [CLRS09] classify sorting algorithms primarily by their theoritical complexity: the quadratic $O(n^2)$ class (Bubble, Selection, Insertion) versus the $O(n \log n)$ class (Merge, Quick, Heap, TimSort). This classification is correct and essential, yet it skips over important factors such as, memory access, branch prediction, and what the input data actually looks like, all of which substantially influence the time a program actually takes to run.

This paper answer a practical question *how does theoretical complexity translate to real execution time on a modern CPU, across a variety of input types and sizes?* This is not just an academic curiosity. A developer choosing between algorithms for a production system must understand not just big-O bounds, but at what input size those bounds become the determining factor, how adaptive algorithms behave on partially ordered data, and which algorithms simply cannot handle the data type at hand.

Informal Description of the Experiment

To answer the question above, we coded nine different sorting algorithms in Python 3 and ran benchmarks across a bunch of configurations. We tested input sizes starting at $n = 50$ (where even quadratic algorithms are instant) to $n = 10\,000\,000$ (where constant factors and memory bandwidth become important). Seven different input were tested: random integers, already-sorted integers, reverse-sorted integers, almost-sorted integers, integer arrays with low cardinality ("flat"), uniformly random floating-point numbers, and uniformly random strings. The goal was to see how 'aware' each algorithm is of the data it's actually sorting.

A key decision is the handling of slow algorithms at large sizes. Rather than allowing a single run to consume hours, we set a *10-second hard timeout* for every test: any run still executing after 10 seconds is killed and recorded as `TIMEOUT`. From practical side, if an algorithm cannot sort 10 000 random integers within 10 seconds is not usable in a real-time system. On the other side, at small sizes ($n \leq 1000$), we repeated each measurement thousands of times and reported the mean elapsed time.

A Running Example

For showing exactly how this works, we will follow the results of a list of $n = 10$ integers $[7, 3, 9, 1, 5, 8, 2, 6, 4, 0]$. Bubble Sort will scan this list up to nine times, performing up to 45 comparisons. Merge Sort will divide it into single lists and merge them in $\lceil \log_2 10 \rceil = 4$ levels, performing at most 24 comparisons. At this tiny scale, both approaches finish in microseconds. As we scale to millions of elements, the difference between $O(n^2)$ and $O(n \log n)$ becomes the difference between unusable and fast.

Contributions

The contributions of this paper are:

1. A reusable benchmark tool written in Python that uses multiprocessing to handle the 10-second timeouts.
2. Test results that prove and measure exactly where $O(n^2)$ algorithms hit a "performance wall" (around $n = 10\,000$ given our 10 sec limit).
3. An analysis across diffrent data types, showing how bubble sort can actually become *linear* $O(n)$ on sorted input while Selection Sort remains $O(n^2)$ regardless of order.
4. A real-world comparison of TimSort against Merge and Heap Sort. This shows the massive speed advantage of using a built-in, C-optimized algorithm over pure Python code, even when they are in the same complexity class.

Paper Structure

The remainder of the paper is organized as follows. Section 2 defines the sorting problem formally and summarises the complexity of each algorithm. Section 3 describes the implementation, benchmark harness, and experimental setup. Section 4 presents and analyses the experimental results, including the data table and cross-profile comparisons. Section 5 situates our findings in the context of prior work. Section 6 summarises the findings and outlines future directions.

2 Formal Description of the Problem and Algorithms

2.1 The Sorting Problem

Definition 1 (Sorting Problem)**.** *Let $A = \langle a_1, a_2, \ldots, a_n \rangle$ be a sequence of n elements drawn from a totally ordered set $(U, \leq)$. A sorting algorithm produces a permutation $A' = \langle a_{\pi(1)}, a_{\pi(2)}, \ldots, a_{\pi(n)} \rangle$ of A such that $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$.*

This formulation, standard in the literature [CLRS09], admits anything can be out in specific order U: integers, floating-point numbers, strings ordered alphabetically, etc. The totally ordered part is significant: it excludes Counting Sort from being applicable to arbitrary types, as will be seen in Section 4.

2.2 Complexity Summary

Table 1 summarises the time and space complexity of the nine algorithms studied. All complexities use n to denote the number of elements. The notation k in Counting Sort refers to the range of values: $k = \max(A) - \min(A)$.

Table 1: Theoretical complexity of the nine sorting algorithms. Best, average, and worst-case time complexities are given in O-notation. Space complexity refers to auxiliary memory beyond the input array.

Algorithm	**Best**	**Average**	**Worst**	**Space**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$
Linked-List Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
TimSort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

2.3 Key Algorithmic Properties

The $O(n^2)$ algorithms share a similar structure: their inner loop performs work based on the *current unsortedness* of the input. For Bubble Sort and Insertion Sort, this is a good thing, means they handle nearly-sorted data types well, (best case $O(n)$), while Selection Sort always performs $\frac{n(n-1)}{2}$ comparisons regardless of input order [CLRS09].

Among the $O(n \log n)$ algorithms, Merge Sort [CLRS09] achieves guaranteed worst-case performance at the cost of $O(n)$ extra memory. Quick Sort, as described in [CLRS09], achieves excellent average-case performance with $O(\log n)$ for stack; our implementation uses a median-of-three strategy to reduce the probability of worst-case $O(n^2)$ behaviour. Heap Sort is in-place and asymptotically optimal but has poor cache performance due to non-sequential memory access during sift-down operations.

TimSort [Peters02] is a hybrid algorithm combining Merge Sort with Insertion Sort. It identifies already-sorted *runs* in the input and merges them. On fully sorted input, TimSort operates in $O(n)$ time because it just sees one giant run; this property is directly observable in our experimental results.

Counting Sort operates in $O(n + k)$ time, where k is the value range. When $k = O(n)$, this is linear and faster than any comparison-based algorithm. However, Counting Sort requires integer keys and a bounded range; it is fundamentally inapplicable to floating-point or string data.

3 Implementation and Experimental Setup

3.1 Implementation Details

All nine algorithms were implemented in Python 3.11, running on an Apple M1 system with 16 GB of RAM under macOS. Using a single language for all algorithms ensures that the measurements reflect algorithmic differences rather than inter-language implementation effects. Each algorithm operates on a fresh copy of the input array, sso one algorithm's sorting didn't accidentally help the next one.

There are two specific details worth mentioning:

The Recursion Limit: Python's default recursion limit of 1000 frames would cause a `RecursionError` when Merge Sort or Quick Sort hit large datasets. The benchmark therefore sets `sys.setrecursionlimit(300_000)`,so the stack had plenty of room for all our test sizes.

Reproducibility: a global `SEED = 42` is passed to Python's `random` module at startup, ensuring that the exact same input arrays are generated in every run of the benchmark. This makes the results fully reproducible.

We also included a Linked-List Merge Sort. This is the same Merge Sort logic, but we first convert the array into a singly-linked list, sort it by moving nodes around, and then turn it back into an array at the end. We added this version specifically to see the "hidden" cost of following pointers compared to working with a standard contiguous array. It's a great way to isolate how much the data structure itself affects the speed.

3.2 Benchmark Harness and Timeout Mechanism

A main challenge in benchmarking algorithms with different performance profiles is that a standard loop would stuck on the first slow algorithm encountered at large sizes. We address this with a *hard-timeout harness* based on Python's `multiprocessing` module:

```python
def benchmark_with_timeout(func, gen, n, runs, timeout=10):
    manager = mp.Manager()
    return_dict = manager.dict()
    p = mp.Process(target=_worker_process,
                   args=(func, gen, n, runs, return_dict))
    p.start()
    p.join(timeout)
    if p.is_alive():
        p.terminate()
        p.join()
        return "TIMEOUT"
    return return_dict.get('result', "ERR")
```

Listing 1: Handling Timeouts using multiprocessing.

This code runs every single test (the specific algorithm, data type, and size) in its own separate process. If a test takes longer than 10 seconds, the script forcefully stops it using `p.terminate()` and the result is recorded as `TIMEOUT`. Doing it this way solves two main problems: first, one really slow sorting run won't freeze the rest of the measurements. Second, because the operating system itself is handling the kill switch, we bypass Python-specific delays like the Global Interpreter Lock (GIL) or garbage collection getting in the way of our timing.

3.3 Input Distributions and Sizes

Seven different data distributions were designed to stress-test different aspects of each algorithm's behavior.

Random Integers drawn randomly from $[0, 100\,000]$. To represent the standard "average case".

Sorted The list $0, 1, 2, \ldots, n-1$. Reveals best-case behaviour for adaptive algorithms.

Reverse Sorted The list $n, n-1, \ldots, 1$. Worst case for Bubble and Insertion Sort.

Almost Sorted A sorted list with 2% of elements randomly swapped. Intermediate case for adaptive algorithms.

Flat These integers from $[0, 5]$, creating arrays with high repetition. Tests behaviour with many duplicate keys.

Random Float Floating-point values in $[0, 10^5)$. Excludes integer-only algorithms such as Counting Sort.

Random String Six-character lowercase ASCII strings. Tests alphabetical comparison and further excludes Counting Sort.

The seven sizes tested are $n \in \{50, 100, 1000, 10\,000, 100\,000, 1\,000\,000, 10\,000\,000\}$. For small sizes ($n \leq 1000$), the benchmark runs each algorithm 100 to 20 000 times and reports the mean, to average out system noise. For large sizes ($n \geq 100\,000$), a single timed run was usually enough because the time it takes is so large that small variations don't really matter.

4 Experimental Results and Analysis

4.1 The $O(n^2)$ Performance Wall

The most striking result of the experiment is how all all three $O(n^2)$ algorithms hit a performance wall at $n = 10\,000$. On the Random integer tests, Bubble Sort completes in $275\,\mu$s at $n = 100$ then jump to 29.97 ms at $n = 1000$ the size went up by $10\times$ but the time went by $100\times$, exactly matching the quadratic prediction. At $n = 10\,000$, Bubble Sort exceeds the 10-second timeout without completing. Insertion Sort and Selection Sort did the same performance: both finish at $n = 1000$ (12.02 ms and 12.73 ms, respectively) and both time out at $n = 10\,000$.

This result confirm that in real-world performance, that we cannot use the $O(n^2)$ algorithms for sorting more than few thousands elements. Even at $n = 1000$, the 12–30 ms range is already borderline for fast applications. A developer relying on Bubble Sort for a dataset that occasionally grows to 10 000 items would observe sudden 10-second hang, which is a huge fail.

There is, a notable exception within the $O(n^2)$ family. On *already-sorted* data, Bubble Sort degrades to $O(n)$ due to its early-exit check. Our results confirm this: on the Sorted profile, Bubble Sort finished $n = 10\,000\,000$ in only 407 ms. Insertion Sort behaves similarly: 624 ms at $n = 10\,000\,000$ on sorted input. But, Selection Sort, which has no such adaptive mechanism—times out at $n = 10\,000$ even on sorted data, confirming its unchanging $\Theta(n^2)$ complexity.

4.2 Performance of $O(n \log n)$ Algorithms

Table 2 presents the measured execution times for three representative algorithms on the Random integer distribution at the three most diagnostically informative sizes.

Table 2: Execution time on the *Random integer* tests for three algorithms at three input sizes. `TIMEOUT` indicates the run exceeded the 10-second limit without completing.

Algorithm	$n = 100$	$n = 10\,000$	$n = 1\,000\,000$
Bubble Sort	$275.67\,\mu s$	TIMEOUT	TIMEOUT
Merge Sort	$84.00\,\mu s$	$13.90\,ms$	$2.034\,s$
TimSort	$4.70\,\mu s$	$850.13\,\mu s$	$143.58\,ms$

At $n = 100$, Merge Sort is already faster than Bubble Sort by about 3.3×, both finish in less than a millisecond. By $n = 10\,000$, Merge Sort finishes in 13.90 ms, while Bubble Sort does not finish at all. At $n = 1\,000\,000$, Merge Sort needed 2.034 seconds. TimSort, completes the same task in 143.58 ms—a 14× improvement. This huge difference reflects not a difference in asymptotic complexity but in constant factors: TimSort is implemented in C within CPython and is very smart about detecting "runs" of data. My Merge Sort, however, is pure Python, so it has to deal with the "overhead" of calling a function for every single element it processes.

Among the $O(n \log n)$ algorithms at $n = 10^6$ on Random data, the observed ranking is: TimSort (143 ms) < Quick Sort (1.49 s) < Merge Sort (2.03 s) < Linked-List Merge Sort (2.17 s) < Heap Sort (3.09 s). Heap Sort's poor showing despite its optimal worst-case complexity is consistent with known cache-unfriendliness: sift-down operations jump between distant memory locations, stressing the hardware prefetcher.

4.3 Counting Sort and Data-Type Constraints

Counting Sort stands apart from all other algorithms in both its strengths and its limitations. On the Random integer tests at $n = 10^6$, it completes in 74.21 ms—faster than every other algorithm including TimSort. At $n = 10\,000\,000$, it completes in 618.67 ms, while all other general-purpose algorithms either time out or take over a second. This performance happens because of its $O(n + k)$ complexity: for random integers in $[0, 100\,000]$, $k = 100\,000 = O(n)$ when n is large, the algoritms acted like it was linear.

However, on the Random Float and Random String tests, Counting Sort is recorded as `UNSUPPORTED` at every size. This is not a performance failure but a semantic one: Counting Sort requires keys to be non-negative integers with a bounded range, and the algorithm implementation raises a `TypeError` when applied to floating-point or string. This makes Counting Sort a *domain-specific* optimisation rather than a general-purpose tool.

4.4 TimSort on Adaptive Profiles

The coolest result from the whole experiment was seeing how TimSort handled the (Already-Sorted) data tests. At $n = 10\,000\,000$, TimSort completes in 145.01 ms. For context, Quick Sort actually timed out on the exact same test. This really shows how "adaptive" TimSort is: it recognizes that the data is already one long sorted run and just does a quick $O(n)$ scan, rather than performing $O(n \log n)$ comparisons and merges.

On the Almost Sorted data, TimSort similarly outperforms all competitors, completing $n = 10^7$ in under 200 ms, while Merge Sort times out. This confirms that TimSort's advantage isn't just for perfectly sorted lists but extends to any input with some kind of

order to it, This is really useful for real-world data like log files or database tables, which are often already partially sorted rather than being completely random.

5 Related Work

The big-O complexity values I used throughout this paper—especially for Merge, Quick, and Heap Sort—come from the classic textbook by Cormen et al. [CLRS09], which provides formal proofs of correctness and complexity for all three algorithms. The theoretical lower bound of $\Omega(n \log n)$ for comparison-based sorting, also established in [CLRS09], frames the context within which all comparison-based algorithms in this study operate.

TimSort was designed by Tim Peters [Peters02] and eventually became the standard sorting algorithm in Python and Java. Its adaptive ability on nearly-sorted data was studied formally by Auger et al. [ANV15], who showed that TimSort achieves $O(n \log r)$ comparisons where r is the number of runs in the input, a refinement of the standard $O(n \log n)$ bound.

A lot of research has also been done on how hardware affects sorting, especially with GPUs. Satish et al. [Sat09] demonstrated that by using the massive parallelism of a GPU, you can get speeds an order of magnitude faster than a standard CPU. Even though my paper focuses on single-threaded CPU performance, these GPU studies show how important hardware-specific design is. Similarly, Hirschberg [Hir78] early work on parallel sorting helped lead to the idea of reducing the work done for every single element—a concept that shows up in TimSort's run-detection as well.

My study fits into this by providing a benchmark that covers several different algorithms and data distributions on a single platform. It's meant to be directly reproducible for other students or practitioners without needing any specialized high-end hardware.

6 Conclusions and Future Work

This paper written to determine is the theoretical complexity shows the real difference when they are ran on a modern CPU. Looking at the data, the answer is a very clear "yes".

The $O(n^2)$ wall is real, and practically consequential. All three quadratic algorithms (Bubble, Selection, and Insertion Sort) exceed a 10-second time budget at $n = 10\,000$ on random integer input. For datasets of this size or larger, no quadratic algorithm should be used unless the input is guaranteed to be nearly sorted (in which case Insertion Sort can be competitive) or the algorithm is serving as a base case within a hybrid approach.

Among the $O(n \log n)$ algorithms, the recommendation is clear: TimSort, (Python's built-in `sorted()`), should be the default choice. It is 10–20× faster than pure-Python Merge Sort or Heap Sort at the same complexity, . The only time you'd really want to use something else is Counting Sort for massive lists of integers with a small range. In that specific scenario, it hits true linear time and actually outperform TimSort by about 2× at $n = 10^6$.

The following guidelines summarise the practical conclusions of this study:

$n < 100$**:** Any algorithm is acceptable. Insertion Sort's simplicity and low overhead make it a reasonable choice for tiny arrays.

$100 \leq n < 10\,000$**:** Use an $O(n \log n)$ algorithm. TimSort is the default unless implementation constraints require a custom sort.

$n \geq 10\,000$**:** $O(n^2)$ algorithms are unusable in real-time contexts. Use TimSort (general purpose) or Counting Sort (bounded integers only).

Partially ordered input: TimSort is especially well-suited; it is the only tested algorithm that completes $n = 10^7$ sorted integers in under 200 ms.

String or float data: Counting Sort is inapplicable. TimSort is the recommended default.

Moving formard, there are three areas to explore. First, the benchmark to handle multi-theaded and distributed systems based on what Satish et al. [Sat09] found, a GPU-based radix sort would probably beat all the algorithms I tested once the data gets big enough. I also want to look closer at how the Python GIL affects multi-process benchmarks. Finally, I'd like to use hardware performance counters to track cache miss rates for the Linked-List Merge Sort. That would help explain why it had that 7% overhead compared to the array version.

References

[CLRS09] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 3rd edition. MIT Press, 2009.

[Peters02] T. Peters. *TimSort — Description and Python implementation.* Python mailing list, 2002. `https://bugs.python.org/issue5371`

[ANV15] N. Auger, C. Nicaud, and C. Pivoteau. Merge complexity of sorting algorithms. In *Proceedings of the 42nd International Colloquium on Automata, Languages, and Programming (ICALP)*, 2015.

[Sat09] N. Satish, M. Harris, and M. Garland. Designing efficient sorting algorithms for manycore GPUs. In *2009 IEEE International Symposium on Parallel & Distributed Processing*, pages 1–10. IEEE, 2009.

[Hir78] D. S. Hirschberg. Fast parallel sorting algorithms. *Communications of the ACM*, 21(8):657–661, 1978.